



GL BAJAJ
Institute of Technology & Management

[Approved by AICTE, Govt. of India & Affiliated to Dr. APJ
Abdul Kalam Technical University, Lucknow, U.P., India]
Department of Computer Sc. & Engineering (AI),



LAB MANUAL

Course: Object Oriented Programming with Java Lab

Code: BCS452

Session: 2025-26

Semester: 4th

Prepared By:

Ms. Bhumika Nirmohi, Asst. Professor CSE-AI Dept

BCS452- Object Oriented Programming with Java

List of Experiments (Indicative & not limited to)

PRACTICAL LIST

S. No.	List of Programs	UNIT	CO
1.	Use Java compiler and eclipse platform to write and execute java programs.	UNIT I	CO1
2.	Creating simple java programs using command line arguments.	UNIT I	CO1
3.	Understand OOP concepts and basics of Java programming (Classes, Objects, Methods).	UNIT I	CO2
4.	Create Java programs using inheritance and polymorphism.	UNIT I	CO2
5.	Implement error-handling techniques using exception handling and multithreading.	UNIT II	CO3
6.	Create a java program with the use of java packages.	UNIT II	CO3
7.	Construct a java program using Java I/O package (Reading/Writing Files).	UNIT III	CO3
8.	Create an industry-oriented application using Spring Framework (Dependency Injection/IoC).	UNIT V	CO4
9.	Test RESTful web services using Spring Boot (GET, POST, PUT, DELETE APIs).	UNIT V	CO4
10.	Test Frontend web application with Spring Boot.	UNIT V	CO4

LAB PLAN

Object oriented with Java Programming Lab(BCS 452)

Exp. No.	Content	Sub-Experiments to be Performed	Lab Turn	Unit / CO
1	Environment & Basic Syntax	1.1 Install JDK and configure Environment Variables. 1.2 Write and compile a "Hello World" program in Eclipse. 1.3 Create a program to display the default values of all primitive data types.	Turn-01	UNIT I / CO1
2	Command Line Arguments	2.1 Write a program to accept two numbers via <code>args[]</code> and display their sum. 2.2 Create a program to take a Username via command line and print a personalized greeting.	Turn-02	UNIT I / CO1

3	Classes, Objects & Methods	3.1 Define a Box class with dimensions; instantiate multiple objects. 3.2 Implement a method calculateVolume() within the class. 3.3 Demonstrate Constructor Overloading (Default vs Parameterized).	Turn-03	UNIT I / CO2
4	Inheritance & Polymorphism	4.1 Implement Single Inheritance (e.g., Animal → Dog). 4.2 Implement Multilevel Inheritance (e.g., Shape → Rectangle → Square). 4.3 Demonstrate Method Overriding for dynamic polymorphism.	Turn-04	UNIT I / CO2
5	Error-Handling & Multithreading	5.1 Use try-catch-finally to handle ArrayIndexOutOfBoundsException. 5.2 Create two threads by extending the Thread class. 5.3 Implement the Runnable interface for a third thread.	Turn-05	UNIT II / CO3

6	Java Packages	6.1 Create a user-defined package named <code>com.calculator.math</code>. 6.2 Add a class with basic arithmetic methods to the package. 6.3 Import and use this package in a separate <code>Main</code> class.	Turn-06	UNIT II / CO3
7	Java I/O (File Handling)	7.1 Use <code>FileWriter</code> to create a file and write a string to it. 7.2 Use <code>FileReader</code> and <code>BufferedReader</code> to read the file content. 7.3 Implement a program to copy content from <code>source.txt</code> to <code>dest.txt</code>.	Turn-07	UNIT III / CO3
8	Spring Framework (DI/IOC)	8.1 Setup a Maven/Gradle project with Spring dependencies. 8.2 Implement Setter Injection and Constructor Injection. 8.3 Use <code>ApplicationContext</code> to retrieve beans from the IoC container.	Turn-08	UNIT V / CO4

9	RESTful Web Services	9.1 Create a @RestController in a Spring Boot application. 9.2 Map GET and POST requests to handle user data. 9.3 Implement PUT and DELETE endpoints to simulate a CRUD app.	Turn-09	UNIT V / CO4
10	Frontend & API Integration	10.1 Create a simple HTML/Thymeleaf frontend for the Spring app. 10.2 Integrate the Frontend with the REST endpoints created in Exp 9. 10.3 Perform end-to-end testing of the application in a browser.	Turn-10	UNIT V / CO4

COURSE OBJECTIVES:

- ☐ To write programs using abstract classes.
- ☐ To write programs for solving real world problems using java collection frame work.
- ☐ To write multithreaded programs.
- ☐ To write GUI programs using swing controls in Java.
- ☐ To introduce java compiler and eclipse platform.



- ☐ To impart hands-on experience with java programming.

COURSE OUTCOMES:

- ☐ Able to write programs for solving real world problems using java collection frame work.
- ☐ Able to write programs using abstract classes.
- ☐ Able to write multithreaded programs.
- ☐ Able to write GUI programs using swing controls in Java.



JAVA LABORATORY MANUAL **COMPLETE EXPERIMENT MATERIALS**

EXP. NO. 1: Use Java compiler and eclipse platform to write and execute java programs.

Lab Turn: Turn-01 | Unit/CO: UNIT I / CO1

1.1 Install JDK and configure Environment Variables

****Steps:****

1. Download JDK 17+ from oracle.com
2. Install and set JAVA_HOME = C:\Program Files\Java\jdk-17
3. Add %JAVA_HOME%\bin to PATH
4. Verify: javac -version & java -version

****Output:****

C:\> javac -version

javac 17.0.2

C:\> java -version

java version "17.0.2"

1.2 Write and compile a "Hello World" program in Eclipse

File: HelloWorld.java

PROGRAM:

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello, World!");  
  
        System.out.println("Welcome to Java Programming Lab");  
  
    }  
  
}
```

OUTPUT:

Hello, World!

Welcome to Java Programming Lab

1.3 Create a program to display default values of all primitive data types

File: [DefaultValues.java](#)

PROGRAM:

```
public class DefaultValues {  
  
    static byte b;  
  
    static short s;  
  
    static int i;  
  
    static long l;  
  
    static float f;  
  
    static double d;  
  
    static char c;  
  
    static boolean bool;
```

```
public static void main(String[] args) {  
    System.out.println("Default values of primitive data types:");  
    System.out.println("byte: " + b);  
    System.out.println("short: " + s);  
    System.out.println("int: " + i);  
    System.out.println("long: " + l);  
    System.out.println("float: " + f);  
    System.out.println("double: " + d);  
    System.out.println("char: '" + c + "' (ASCII: " + (int)c + ")");  
    System.out.println("boolean: " + bool);  
}  
}
```

OUTPUT:

Default values of primitive data types:

byte: 0

short: 0

int: 0

long: 0

float: 0.0

double: 0.0

char: ' ' (ASCII: 0)

boolean: false

EXP. NO. 2: Creating simple java programs using command line arguments.

Lab Turn: Turn-02 | Unit/CO: UNIT I / CO1

2.1 Write a program to accept two numbers via args[] and display their sum

File: [SumArgs.java](#)

PROGRAM:

```
public class SumArgs {  
  
    public static void main(String[] args) {  
  
        if (args.length != 2) {  
  
            System.out.println("Usage: java SumArgs <num1> <num2>");  
  
            return;  
  
        }  
  
  
        try {  
  
            int num1 = Integer.parseInt(args[0]);  
  
            int num2 = Integer.parseInt(args[1]);  
  
            System.out.println("Sum of " + num1 + " + " + num2 + " = " + (num1 + num2));  
  
        } catch (NumberFormatException e) {  
  
            System.out.println("Please enter valid integers!");  
  
        }  
  
    }  
}
```

```
}  
  
}  
  
}
```

Command: `java SumArgs 25 35`

OUTPUT :

Sum of 25 + 35 = 60

2.2 Create a program to take Username via command line and print personalized greeting

File: `Greeting.java`

PROGRAM:

```
public class Greeting {  
  
    public static void main(String[] args) {  
  
        if (args.length == 0) {  
  
            System.out.println("Usage: java Greeting <username>");  
  
            return;  
  
        }  
  
  
        String username = args[0];  
  
        System.out.println("Hello " + username + "!");  
  
        System.out.println("Welcome to Java Programming Laboratory!");  
  
    }  
  
}
```

```
}
```

Command: `java Greeting Alice`

OUTPUT:

Hello Alice!

Welcome to Java Programming Laboratory!

EXP. NO. 3: Classes, Objects & Methods

Lab Turn: Turn-03 | Unit/CO: UNIT I / CO2

3.1 Define a Box class with dimensions; instantiate multiple objects

File: `BoxDemo.java`

PROGRAM:

```
class Box {  
    double length, width, height;  
  
    Box(double l, double w, double h) {  
        length = l;  
        width = w;  
        height = h;  
    }  
  
    void displayDimensions() {  
        System.out.printf("Box Dimensions: %.2f x %.2f x %.2f\n", length, width, height);  
    }  
}
```

```
public class BoxDemo {  
    public static void main(String[] args) {  
        Box box1 = new Box(10.5, 5.2, 8.0);  
        Box box2 = new Box(15.0, 7.5, 12.3);  
  
        System.out.println("Box 1:");  
        box1.displayDimensions();  
        System.out.println("Box 2:");  
        box2.displayDimensions();  
    }  
}
```

OUTPUT:

Box 1:

Box Dimensions: 10.50 x 5.20 x 8.00

Box 2:

Box Dimensions: 15.00 x 7.50 x 12.30

3.2 Implement calculateVolume() method

File: [BoxVolume.java](#)

PROGRAM:

```
class Box {  
    double length, width, height;
```

```
Box(double l, double w, double h) {  
    length = l; width = w; height = h;  
}
```

```
double calculateVolume() {  
    return length * width * height;  
}
```

```
void display() {  
    System.out.printf("Volume: %.2f cubic units\n", calculateVolume());  
}  
}
```

```
public class BoxVolume {  
    public static void main(String[] args) {  
        Box box = new Box(12, 8, 6);  
        box.display();  
    }  
}
```

OUTPUT:

Volume: 576.00 cubic units

3.3 Demonstrate Constructor Overloading

File: `BoxConstructor.java`

PROGRAM:

```
class Box {  
  
    double length, width, height;  
  
    // Default constructor  
    Box() {  
        length = width = height = 1.0;  
        System.out.println("Default constructor called");  
    }  
  
    // Parameterized constructor  
    Box(double side) {  
        length = width = height = side;  
        System.out.println("Single parameter constructor called");  
    }  
  
    Box(double l, double w, double h) {  
        length = l; width = w; height = h;  
        System.out.println("Three parameter constructor called");  
    }  
}
```



```
}  
  
double volume() {  
    return length * width * height;  
}  
  
}  
  
public class BoxConstructor {  
    public static void main(String[] args) {  
        Box b1 = new Box();        // Default  
        Box b2 = new Box(5.0);     // Single param  
        Box b3 = new Box(2, 4, 6); // Three params  
  
        System.out.printf("b1 volume: %.2f\n", b1.volume());  
        System.out.printf("b2 volume: %.2f\n", b2.volume());  
        System.out.printf("b3 volume: %.2f\n", b3.volume());  
    }  
}
```

OUTPUT:

Default constructor called

Single parameter constructor called

Three parameter constructor called



b1 volume: 1.00

b2 volume: 125.00

b3 volume: 48.00

EXP. NO. 4: Inheritance & Polymorphism

Lab Turn: Turn-04 | Unit/CO: UNIT I / CO2

4.1 Implement Single Inheritance (Animal → Dog)

File: [SingleInheritance.java](#)

PROGRAM:

```
class Animal {  
    String name;  
  
    Animal(String name) {  
        this.name = name;  
    }  
  
    void eat() {  
        System.out.println(name + " is eating");  
    }  
}
```

```
void sleep() {  
    System.out.println(name + " is sleeping");  
}  
}
```

```
class Dog extends Animal {  
    Dog(String name) {  
        super(name);  
    }  
}
```

```
void bark() {  
    System.out.println(name + " is barking");  
}  
}
```

```
public class SingleInheritance {  
    public static void main(String[] args) {  
        Dog dog = new Dog("Tommy");  
        dog.eat();  
        dog.sleep();  
        dog.bark();  
    }  
}
```

```
}  
}
```

OUTPUT:

Tommy is eating
Tommy is sleeping
Tommy is barking

4.2 Implement Multilevel Inheritance (Shape → Rectangle → Square)

File: [MultilevelInheritance.java](#)

PROGRAM:

```
class Shape {  
    String color;  
  
    Shape(String color) {  
        this.color = color;  
    }  
  
    void displayColor() {  
        System.out.println("Color: " + color);  
    }  
}  
  
class Rectangle extends Shape {
```

double length, width;

Rectangle(String color, double l, double w) {

 super(color);

 length = l;

 width = w;

}

double area() {

 return length * width;

}

}

class Square extends Rectangle {

 Square(String color, double side) {

 super(color, side, side);

 }

void display() {

 super.displayColor();

 System.out.printf("Square side: %.2f, Area: %.2f\n", length, area());

}

```
}
```

```
public class MultilevelInheritance {  
    public static void main(String[] args) {  
        Square sq = new Square("Red", 5.5);  
        sq.display();  
    }  
}
```

OUTPUT:

Color: Red

Square side: 5.50, Area: 30.25

4.3 Demonstrate Method Overriding

File: [MethodOverriding.java](#)

PROGRAM:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks: Woof!");  
    }  
}
```

```
class Cat extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Cat meows: Meow!");  
    }  
}
```

```
public class MethodOverriding {  
    public static void main(String[] args) {  
        Animal[] animals = {new Dog(), new Cat()};  
  
        for (Animal a : animals) {  
            a.sound(); // Dynamic polymorphism  
        }  
    }  
}
```



}

OUTPUT:

Dog barks: Woof!

Cat meows: Meow!

EXP. NO. 5: Error-Handling & Multithreading

Lab Turn: Turn-05 | Unit/CO: UNIT II / CO3

5.1 Use try-catch-finally for ArrayIndexOutOfBoundsException

File: [ErrorHandling.java](#)

PROGRAM:

```
public class ErrorHandling {  
    public static void main(String[] args) {  
        try {  
            int[] arr = {10, 20, 30};  
            System.out.println("Element at index 5: " + arr[5]);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Error: " + e.getMessage());  
            System.out.println("Array length: " + 3);  
        } finally {  
            System.out.println("Finally block executed");  
        }  
    }  
}
```

OUTPUT:

Error: Index 5 out of bounds for length 3

Array length: 3

Finally block executed

5.2 Create two threads by extending Thread class

File: ThreadExtend.java

PROGRAM:

```
class MyThread1 extends Thread {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Thread1: " + i);  
            try { Thread.sleep(500); } catch (InterruptedException e) {}  
        }  
    }  
}  
  
class MyThread2 extends Thread {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Thread2: " + i);  
            try { Thread.sleep(500); } catch (InterruptedException e) {}  
        }  
    }  
}  
  
public class ThreadExtend {  
    public static void main(String[] args) {  
        MyThread1 t1 = new MyThread1();
```

```
MyThread2 t2 = new MyThread2();

t1.start();

t2.start();

}

}
```

OUTPUT (Sample - order may vary):

```
Thread1: 1
Thread2: 1
Thread1: 2
Thread2: 2
Thread1: 3
Thread2: 3
...
```

5.3 Implement Runnable interface

File: [RunnableDemo.java](#)

PROGRAM:

```
class MyRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Runnable Thread: " + i);
            try { Thread.sleep(500); } catch (InterruptedException e) {}
        }
    }
}

public class RunnableDemo {
```



```
public static void main(String[] args) {  
    Thread t3 = new Thread(new MyRunnable());  
    t3.start();  
}  
}
```

OUTPUT:

Runnable Thread: 1
Runnable Thread: 2
Runnable Thread: 3
Runnable Thread: 4
Runnable Thread: 5

EXP. NO. 6: Create a java program with the use of java packages.

Lab Turn: Turn-06 | Unit/CO: UNIT II / CO3

6.1-6.3 Complete Package Implementation

Directory Structure:

```
com/  
  calculator/  
    math/  
      Calculator.java  
MainClass.java (in root)
```

File: [com/calculator/math/Calculator.java](#)

```
package com.calculator.math;  
  
public class Calculator {  
    public static int add(int a, int b) {  
        return a + b;  
    }  
  
    public static int subtract(int a, int b) {  
        return a - b;  
    }  
  
    public static int multiply(int a, int b) {  
        return a * b;  
    }  
  
    public static double divide(double a, double b) {  
        if (b == 0) return 0.0;  
        return a / b;  
    }  
}
```

```
}  
}
```

File: **MainClass.java**

```
import com.calculator.math.Calculator;  
  
public class MainClass {  
    public static void main(String[] args) {  
        System.out.println("5 + 3 = " + Calculator.add(5, 3));  
        System.out.println("10 - 4 = " + Calculator.subtract(10, 4));  
        System.out.println("6 * 7 = " + Calculator.multiply(6, 7));  
        System.out.println("15 / 3 = " + Calculator.divide(15, 3));  
    }  
}
```

Compile & Run:

```
javac -d . com/calculator/math/Calculator.java  
javac -cp . MainClass.java  
java MainClass
```

OUTPUT:

```
5 + 3 = 8  
10 - 4 = 6  
6 * 7 = 42  
15 / 3 = 5.0
```

EXP. NO. 7: Java I/O (File Handling)

Lab Turn: Turn-07 | Unit/CO: UNIT III / CO3

7.1 Use FileWriter to create and write file

File: `FileWriterDemo.java`

PROGRAM:

```
import java.io.FileWriter;
import java.io.IOException;

public class FileWriterDemo {
    public static void main(String[] args) {
        try (FileWriter fw = new FileWriter("output.txt")) {
            fw.write("Hello Java File Handling!\n");
            fw.write("This is line 2.\n");
            fw.write("Experiment 7.1 completed.");
            System.out.println("File written successfully!");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Output: `output.txt` created with content

7.2 Use FileReader and BufferedReader

File: `FileReaderDemo.java`

PROGRAM:

```
import java.io.*;

public class FileReaderDemo {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new FileReader("output.txt"))) {
            String line;
            System.out.println("File content:");
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

OUTPUT:

File content:
Hello Java File Handling!
This is line 2.
Experiment 7.1 completed.

7.3 Copy content from source.txt to dest.txt

File: **FileCopy.java**

PROGRAM:

```
import java.io.*;

public class FileCopy {
    public static void main(String[] args) {
```



```
try (BufferedReader br = new BufferedReader(new FileReader("output.txt"));  
    FileWriter fw = new FileWriter("dest.txt")) {  
  
    String line;  
    while ((line = br.readLine()) != null) {  
        fw.write(line + "\n");  
    }  
    System.out.println("File copied successfully!");  
} catch (IOException e) {  
    e.printStackTrace();  
}  
}
```

OUTPUT:

File copied successfully!

EXP. NO. 8: Spring Framework (DI/IOC)

Lab Turn: Turn-08 | Unit/CO: UNIT V / CO4

Complete Maven Project Structure

pom.xml

src/main/java/

com/lab/

AppConfig.java

Student.java

MainApp.java

File: pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.lab</groupId>
  <artifactId>spring-di</artifactId>
  <version>1.0</version>

  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>5.3.21</version>
    </dependency>
  </dependencies>
</project>
```

File: [src/main/java/com/lab/Student.java](#)

```
package com.lab;

public class Student {
    private String name;
    private int age;
    private String course;

    // Setter Injection
    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }
    public void setCourse(String course) { this.course = course; }

    // Constructor Injection
    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void display() {
        System.out.println("Student: " + name + ", Age: " + age + ", Course: " + course);
    }
}
```

File: [src/main/java/com/lab/AppConfig.java](#)

```
package com.lab;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {
```

```
@Bean(name="student1") // Setter Injection
public Student setterStudent() {
    Student student = new Student("Alice", 20);
    student.setCourse("Java");
    return student;
}
```

```
@Bean(name="student2") // Constructor Injection
public Student constructorStudent() {
    return new Student("Bob", 22);
}
}
```

File: [src/main/java/com/lab/MainApp.java](#)

```
package com.lab;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class MainApp {
    public static void main(String[] args) {
        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);

        Student s1 = context.getBean("student1", Student.class);
        Student s2 = context.getBean("student2", Student.class);

        System.out.println("=== Setter Injection ===");
        s1.display();

        System.out.println("\n=== Constructor Injection ===");
        s2.setCourse("Spring");
        s2.display();
    }
}
```



```
}  
}
```

Maven Commands:

```
mvn clean compile exec:java -Dexec.mainClass="com.lab.MainApp"
```

OUTPUT:

```
=== Setter Injection ===
```

```
Student: Alice, Age: 20, Course: Java
```

```
=== Constructor Injection ===
```

```
Student: Bob, Age: 22, Course: Spring
```

EXP. NO. 9: RESTful Web Services

Lab Turn: Turn-09 | Unit/CO: UNIT V / CO4

Complete Spring Boot Project

File: `pom.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.lab</groupId>
  <artifactId>rest-crud</artifactId>
  <version>1.0</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.7.5</version>
  </parent>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>
</project>
```

File: `src/main/java/com/lab/User.java`

```
package com.lab;

public class User {
  private Long id;
```

```
private String name;  
private String email;
```

```
// Constructors
```

```
public User() {}  
public User(Long id, String name, String email) {  
    this.id = id;  
    this.name = name;  
    this.email = email;  
}
```

```
// Getters & Setters
```

```
public Long getId() { return id; }  
public void setId(Long id) { this.id = id; }  
public String getName() { return name; }  
public void setName(String name) { this.name = name; }  
public String getEmail() { return email; }  
public void setEmail(String email) { this.email = email; }  
}
```

File: [src/main/java/com/lab/UserController.java](#)

```
package com.lab;
```

```
import java.util.HashMap;  
import java.util.Map;  
import org.springframework.web.bind.annotation.*;
```

```
@RestController
```

```
@RequestMapping("/api/users")
```

```
public class UserController {
```

```
    private Map<Long, User> users = new HashMap<>();  
    private static Long counter = 1L;
```

// 9.2 GET - Get all users

@GetMapping

```
public Map<Long, User> getAllUsers() {  
    return users;  
}
```

// 9.2 POST - Create user

@PostMapping

```
public User createUser(@RequestBody User user) {  
    user.setId(counter++);  
    users.put(user.getId(), user);  
    return user;  
}
```

// 9.3 PUT - Update user

@PutMapping("/{id}")

```
public User updateUser(@PathVariable Long id, @RequestBody User userDetails) {  
    User user = users.get(id);  
    if (user != null) {  
        user.setName(userDetails.getName());  
        user.setEmail(userDetails.getEmail());  
    }  
    return user;  
}
```

// 9.3 DELETE - Delete user

@DeleteMapping("/{id}")

```
public String deleteUser(@PathVariable Long id) {  
    users.remove(id);  
    return "User with ID " + id + " deleted successfully";  
}  
}
```

File: [src/main/java/com/lab/RestApplication.java](#)


```
package com.lab;
```

```
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication  
public class RestApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(RestApplication.class, args);  
    }  
}
```

Run: mvn spring-boot:run

Test Endpoints (using curl/Postman):

GET http://localhost:8080/api/users

POST http://localhost:8080/api/users

Body: {"name":"John","email":"john@test.com"}

PUT http://localhost:8080/api/users/1

Body: {"name":"John Updated","email":"john.new@test.com"}

DELETE http://localhost:8080/api/users/1

Sample Output:

GET Response: {1: {id:1, name:"John", email:"john@test.com"}}

POST Response: {id:1, name:"John", email:"john@test.com"}

DELETE Response: "User with ID 1 deleted successfully"

EXP. NO. 10: Frontend & API Integration

Lab Turn: Turn-10 | Unit/CO: UNIT V / CO4

10.1-10.3 Complete Full-Stack Application

Add to pom.xml (Spring Boot Project from Exp 9):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

File: [src/main/resources/templates/index.html](#)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>User Management System</title>
  <meta charset="UTF-8">
  <style>
    body { font-family: Arial; margin: 40px; }
    table { border-collapse: collapse; width: 100%; }
    th, td { border: 1px solid #ddd; padding: 8px; text-align: left; }
    th { background-color: #f2f2f2; }
    button { padding: 5px 10px; margin: 2px; }
    input { padding: 5px; margin: 2px; }
  </style>
</head>
<body>
  <h1>User Management System</h1>

  <!-- Create User Form -->
  <h3>Add New User</h3>
  <form th:action="@{/api/users}" method="post">
```

```
<input type="text" name="name" placeholder="Name" required>
<input type="email" name="email" placeholder="Email" required>
<button type="submit">Add User</button>
</form>

<!-- Users Table -->
<h3>Users List</h3>
<table>
  <tr>
    <th>ID</th>
    <th>Name</th>
    <th>Email</th>
    <th>Actions</th>
  </tr>
  <tr th:each="user : ${users}">
    <td th:text="${user.id}"></td>
    <td th:text="${user.name}"></td>
    <td th:text="${user.email}"></td>
    <td>
      <button onclick="deleteUser(${user.id})">Delete</button>
    </td>
  </tr>
</table>

<script>
  function deleteUser(id) {
    if (confirm('Delete this user?')) {
      fetch('/api/users/' + id, {method: 'DELETE'})
        .then(() => location.reload());
    }
  }
</script>
</body>
</html>
```



Updated Controller: [src/main/java/com/lab/UserController.java](#)

```
// Add this method to existing controller
@GetMapping("/")
public String home(Model model) {
    model.addAttribute("users", users);
    return "index";
}
```

Run: mvn spring-boot:run

Access: <http://localhost:8080>

Output (Browser):

User Management System

[Form to add users]

Users List Table with Add/Delete functionality

End-to-End CRUD operations working in browser